

Archivara Agent: Seeded Cellular-Automaton Falsification on the 64-Modular Order-668 Hadamard Frontier

March 18, 2026

Repository scope. The initiating task asked whether a previously untried “cellar automata” method could solve Hadamard order 668. Neither the repository nor the literature snapshot produced evidence for a distinct method family of that name, so throughout the paper we interpret the phrase as *cellular automata*. The contribution we report is intentionally narrow: a seeded cellular-automaton repair attempt on the published 64-modular order-668 frontier object, executed under matched same-representation controls, together with a direct negative result.

Artifact summary. The manuscript is generated from the saved experiment pack in `results/`, the bibliography in `sources.bib`, and the programmatic figures in `figures/`. The figure set was rendered by `scripts/generate_paper_artifacts.py`; the direct locality artifact used in Sections 5 and 7 is `results/analysis/frontier_locality_scan.json`.

Abstract

We study a sharply delimited question: can a genuinely local cellular-automaton repair dynamic, started from the published 64-modular order-668 near-solution of Eliahou, produce an exact real Hadamard matrix or at least outperform matched non-CA local search in the same compressed coordinates? The repository already contained the frontier seed, a shared exactness harness, and a pre-registered continuation gate. We therefore wrote the paper around direct artifacts rather than around speculative mechanism stories.

The implemented branch, `H1_defect_syndrome_ca_64m`, evolves a compressed state consisting of two binary length-167 sequences, a 166-lag defect field, an active-lag mask, and refractory packet memory. It is benchmarked against greedy search, tabu search, simulated annealing, and stochastic hillclimbing under a shared seed, shared packet basis, shared budget 80, requested restart count 3, and shared exact-hit criterion. The harness is first validated on two exact controls of orders 20 and 28, where H1 reaches exactness in both cases.

The decisive frontier result is negative. On the canonical order-668 seed, no method attains exact orthogonality. H1 never improves the seed best objective (13, 2880, 512) in support, l_1 , and maximum defect magnitude; instead its three frontier restarts terminate at median objective (33, 2356, 384), indicating lower magnitude only after support diffusion. A direct exhaustive one-packet scan over the same seed finds zero improving moves among 334 packets, exactly one neutral symmetry packet, and a median changed-lag footprint of 51 out of 166 lags. This shows that the raw single-packet basis is syntactically local but nonlocal in effect.

The resulting contribution is therefore a *falsification result*: the executed single-packet defect-syndrome CA does not advance the order-668 frontier and does not survive matched controls as a distinct CA repair method. The negative result matters because it isolates the bottleneck precisely. If a future CA branch is to remain credible, it must change the actuator basis or the locality graph rather than merely retune the present rule weights.

1 Introduction

An $n \times n$ real Hadamard matrix is a ± 1 matrix H satisfying

$$HH^\top = nI_n. \tag{1}$$

Hadamard existence is elementary to state, structurally rigid, and algorithmically difficult. Modern databases and surveys make clear that the landscape is a mixture of exact constructions, structured-family searches, and persistent open orders [4, 6]. The case relevant here is order $668 = 4 \cdot 167$: the repository’s exact target is a real Hadamard matrix, while the strongest in-repo frontier anchor is Eliahou’s 2025 64-modular Hadamard matrix of order 668, a near-solution rather than an exact witness [7].

The motivating hypothesis behind this work is narrower than “use cellular automata for Hadamard matrices.” Cellular automata (CA) are already known as generic local search or propagation mechanisms [13], and they also appear in nearby combinatorial-design settings such as cellular-automaton constructions of mutually orthogonal Latin squares and bent-function families [8, 9]. What had not been tested inside the repository was a seed-matched CA *repair* dynamic applied directly to Eliahou’s recovered order-668 seed under the same coordinate system and the same objective accounting as non-CA controls.

The resulting study is therefore best viewed as a pre-registered kill test. We do not ask whether CA are philosophically interesting, or whether near-Hadamard traces can be made visually appealing. We ask a concrete, falsifiable question:

Can a compressed defect-syndrome cellular automaton, operating in the same q/s co-ordinate system as the published 64-modular order-668 seed, reach exact orthogonality or beat matched non-CA local search on the same seed?

The answer delivered by the saved artifacts is no. The negative result is not vacuous: the branch is executable on exact small controls, it shares a validated harness with the baselines, and its failure can be localized. A direct exhaustive one-packet scan on the canonical seed shows that the active single-packet basis is already frozen under the shared proxy objective. The frontier trajectories then reveal the only remaining escape route for H1: support diffusion, which improves l_1 and maximum defect size only by creating more nonzero lags and therefore failing the continuation gate.

Our contributions are correspondingly narrow.

- C1.** We formalize the exact repository representation of the order-668 frontier seed: the length-167 binary sequences, the fixed prime involution, the derived quadruple, and the defect metrics used by the shared harness.
- C2.** We give a complete mathematical derivation of the exact single-packet autocorrelation update used by the CA, and we analyze the asymptotic cost of one H1 update under the implemented code path.
- C3.** We assemble the matched experimental record: two exact controls, one canonical frontier seed, one CA branch, and four non-CA baselines under shared accounting.
- C4.** We add a direct exhaustive one-packet locality scan on the canonical seed, converting the repository’s qualitative “fake locality” concern into a finite computation over all 334 packets.
- C5.** We show that the executed H1 branch fails its first continuation gate and should be read as a falsification scaffold, not as a new CA repair method or an order-668 advance.

The remainder of the paper follows the same discipline. Section 2 positions the work against exact Hadamard search, heuristic search, and CA-adjacent design generation. Section 3 defines the representation and objective precisely. Section 4 describes the harness, baselines, and H1 algorithm. Section 5 proves the packet-delta identity, the implementation-level complexity bound, and the computational freeze propositions. Sections 6 and 7 report the experimental record. Section 8 explains what the negative result does and does not imply.

2 Related Work

2.1 Hadamard and modular-Hadamard context

The repository does not start from a blank search space. It starts from a concrete modular frontier. Eliahou and Kervaire developed the modular-sequence and modular-Hadamard background relevant to this lineage [5, 6]. The specific order-668 seed used here is Eliahou’s 2025 construction of a 64-modular Hadamard matrix obtained from a compact length-167 seed; Fact 3.1 of that paper provides the exact run-length encoding and the 13 exceptional coefficients that define the frontier instance [7]. We therefore make no claim that the present work contributes a new seed, a new exact construction, or a new modular theorem. It is downstream experimentation on top of a published near-solution.

2.2 Exact and heuristic Hadamard search

The exact-search ceiling is set by certificate-rich structured methods, not by unconstrained local heuristics. Bright and coauthors show how SAT+CAS methods can prove exhaustive statements over structured Hadamard-adjacent families such as Williamson matrices and best matrices [2, 3]. Those works matter here as a standard of evidence: they return exact combinatorial statements and certificates, whereas the present CA branch returns neither.

Within heuristic search, Suksmono and collaborators cast Hadamard search as simulated quantum annealing, quantum annealing, and quantum-computing-friendly optimization over binary variables [10–12]. These papers are the correct literature comparators for any broad claim that a new local-search mechanism is competitive on Hadamard search. The repository’s own verification pack correctly rejects that stronger claim: our frontier pilot is seed-matched and fair enough for branch elimination, but it is vastly smaller than those literature-scale comparisons.

Projection and feasibility methods form a different non-CA baseline family. Artacho, Borwein, and Tam analyze Douglas–Rachford feasibility methods for matrix completion problems [1]. We cite this line not because H1 resembles it closely, but because it illustrates another way to phrase matrix-structured search without invoking CA language.

2.3 Cellular automata as search and design machinery

The closest generic CA-as-search comparator in the bibliography is Tsompanas et al.’s shortest-path application [13]. It weakens any broad novelty statement such as “cellular automata for search are new.” At the same time, it is not Hadamard-specific: the state geometry, objective, certification standard, and seed dependence are all different.

More importantly, CA are already present in neighboring combinatorial-design settings. Mariot et al. study mutually orthogonal Latin squares generated from cellular automata, while Gadouleau, Mariot, and Picek construct bent functions from cellular automata [8, 9]. Bent functions are directly Hadamard-adjacent through Walsh spectra, and MOLS construction is another example where CA supply algebraic design structure rather than generic local search. These works make two points that matter for this paper. First, we cannot honestly claim that CA are new in combinatorial design. Second, the only defensible niche left for H1 is much narrower: CA as a seed-specific repair dynamic on a published order-668 frontier object.

2.4 How this paper differs

The repository’s own prior-art gap analysis is unusually explicit, and the final paper preserves that discipline. Relative to Eliahou’s order-668 paper, we contribute no new frontier construction and no exact witness. Relative to Tsompanas et al., we are not presenting a generic CA-for-search story; we are testing one compressed defect-syndrome CA in a rigid exactness framework. Relative to Suksmono’s annealing papers and Bright’s SAT+CAS work, we are far narrower in scope and weaker in certification. Relative to CA-based design-generation papers such as [8, 9], we are not generating a new combinatorial family at all. What survives after all those comparisons is not a broad positive method claim, but a carefully documented negative result.

3 Background and Preliminaries

3.1 The order-668 seed representation

Let $\ell = 167$. The repository works with two binary sequences

$$q, s \in \{\pm 1\}^\ell, \quad (2)$$

stored in the compact q/s coordinate system recovered from [7]. Indices run from 0 to $\ell - 1$. Define the *prime involution*

$$x'_i = \begin{cases} x_i, & 0 \leq i < (\ell + 1)/2, \\ -x_i, & (\ell + 1)/2 \leq i \leq \ell - 1, \end{cases} \quad (3)$$

and the pointwise product $(q \odot s)_i = q_i s_i$.

The derived quadruple is

$$A = s, \quad B = s', \quad C = q \odot s, \quad D = (q \odot s)'. \quad (4)$$

This is exactly the lift implemented in the shared harness and exactly the lift used to decode the published frontier seed [5, 7].

3.2 Aperiodic autocorrelation and defect metrics

Definition 3.1 (Aperiodic autocorrelation). *For $x \in \{\pm 1\}^\ell$ and lag $k \in \{0, 1, \dots, \ell - 1\}$, define*

$$N_x(k) = \sum_{t=0}^{\ell-k-1} x_t x_{t+k}. \quad (5)$$

The combined correlation coefficients of the quadruple in (4) are

$$c_k(q, s) = N_A(k) + N_B(k) + N_C(k) + N_D(k), \quad k = 0, \dots, \ell - 1. \quad (6)$$

By construction $c_0 = 4\ell$. The shared harness reports exactness from the nonzero-lag coefficients $c_1, \dots, c_{\ell-1}$ and independently audits the full Gram defect of the Goethals–Seidel array built from (A, B, C, D) .

Definition 3.2 (Primary defect summary). *For a state (q, s) , define*

$$\text{support}(q, s) = |\{k \in \{1, \dots, \ell - 1\} : c_k(q, s) \neq 0\}|, \quad (7)$$

$$\text{ll}(q, s) = \sum_{k=1}^{\ell-1} |c_k(q, s)|, \quad (8)$$

$$\text{max}(q, s) = \max_{1 \leq k \leq \ell-1} |c_k(q, s)|. \quad (9)$$

The implementation additionally uses the scalar proxy

$$\text{score}(q, s) = 10^6 \text{support}(q, s) + 10^3 \text{ll}(q, s) + \text{max}(q, s) \quad (10)$$

for within-run comparisons. We emphasize that the branch continuation decision is *not* made from (10) alone. The externally reported gate uses exact-hit rate first, then the support–magnitude pair. We therefore report the triple (support, ll, max) directly throughout the paper.

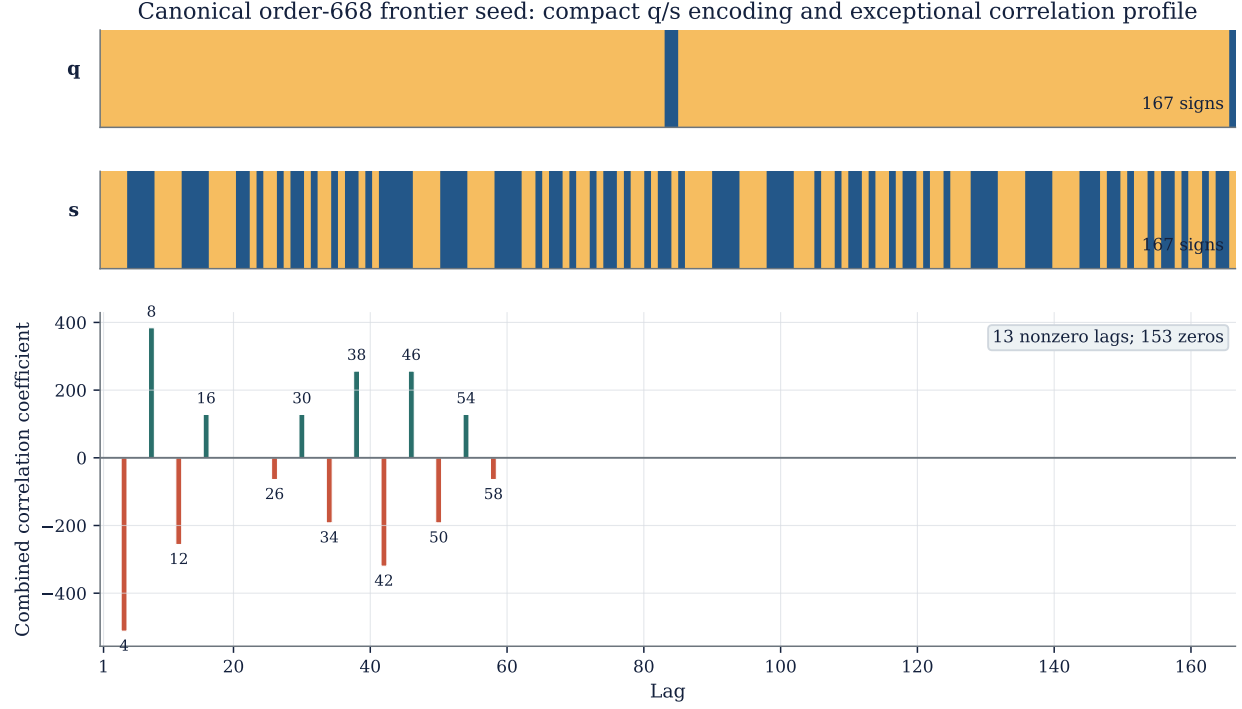


Figure 1: **Recovered order-668 frontier seed.** The upper strips show the exact binary q and s sequences decoded from the published run-length encoding of Eliahou [7, Fact 3.1]. The lower panel shows the combined correlation coefficients c_k from (6). Only 13 of the 166 nonzero-lag coefficients are nonzero; their exact values are listed in (11). This figure is not a search result from the present paper. It is the fixed starting object that every order-668 experiment in this study uses.

3.3 Canonical frontier seed

Eliahou’s published order-668 seed is encoded by the run lengths

$$q : (83, 2, 81, 1), \quad s : (4)^5(2, 1, 1)^5(1, 5)(4)^4(2, 1, 1)^6(4)^4(3)(1, 2, 1)^5(3)(4)^4(3)(1, 2, 1)^5,$$

with starting sign $+1$ [7, Fact 3.1]. Figure 1 visualizes the recovered sequences and the corresponding coefficient profile. The same source gives exactly 13 exceptional nonzero coefficients:

$$\begin{aligned} c_4 &= -512, & c_8 &= 384, & c_{12} &= -256, & c_{16} &= 128, \\ c_{26} &= -64, & c_{30} &= 128, & c_{34} &= -192, & c_{38} &= 256, \\ c_{42} &= -320, & c_{46} &= 256, & c_{50} &= -192, & c_{54} &= 128, \\ c_{58} &= -64. \end{aligned} \tag{11}$$

All other nonzero-lag coefficients vanish. The seed therefore starts at the objective triple

$$(\text{support}, \text{ll}, \text{max}) = (13, 2880, 512). \tag{12}$$

4 Method

4.1 Shared harness and exactness contract

All methods are evaluated through the same harness. Given (q, s) and a search method, the harness computes:

- exact-hit status, defined as zero nonzero-lag correlation defects,
- off-diagonal Gram-defect support and maximum defect magnitude,
- the primary defect triple (support, l1, max),
- wall-clock time, objective evaluations, restart statistics, and a sign-symmetry fingerprint.

The harness is validated on two exact controls of lengths 5 and 7, corresponding to Hadamard orders 20 and 28. In both cases, the validation artifact records zero correlation defects and zero off-diagonal Gram defects, so the exact-hit flag is not inferred from a soft proxy.

This shared contract matters because the repository’s own verification pack rejects broader benchmark language. Our benchmark is fair enough for branch elimination, not for publication-level competitiveness with the full Hadamard-search literature. We therefore keep the scope aligned with what the artifacts actually support.

4.2 Matched non-CA baselines

The non-CA baselines operate in exactly the same state space as H1: a pair of length-167 binary sequences and a one-packet move basis where one move flips exactly one q bit or one s bit.

Greedy search. At each iteration, greedy search evaluates every packet in the shared basis, accepts the single best improving move under (10), and stops if no improving packet exists.

Tabu search. Tabu search evaluates the same packet set but attaches a tenure-5 tabu restriction, allowing tabu moves only under aspiration when the move beats the best state seen so far.

Simulated annealing. Simulated annealing samples one random packet at a time, accepts all improving moves and accepts worsening moves with Metropolis probability $\exp(-\Delta/T)$, with initial temperature 100 and multiplicative cooling factor 0.98.

Stochastic hillclimb. Stochastic hillclimbing samples 6 packets without replacement at each step and accepts the first improving move; if none improves the score, the restart terminates.

Every baseline uses the same nominal budget 80, the same requested restart count 3, the same restart diversification rule of two random packet flips on nonzero restarts, and the same RNG seed 17. The frontier comparison is therefore seed-matched, representation-matched, and budget-matched at the level of the saved configuration files.

4.3 H1 defect-syndrome cellular automaton

The H1 state is deliberately compressed. It does *not* evolve a raw 668×668 sign matrix. The state at any accepted step is

$$(q, s, c, \mathcal{M}, r), \quad (13)$$

where $q, s \in \{\pm 1\}^{167}$, $c \in \mathbb{Z}^{167}$ is the combined correlation vector from (6), $\mathcal{M} \subseteq \{1, \dots, 166\}$ is an active-lag neighborhood around the current defect support, and $r \in \mathbb{Z}_{\geq 0}^{334}$ is refractory memory over the packet lattice.

The packet set is

$$\mathcal{P} = \{0, 1, \dots, 333\}, \quad (14)$$

where packets 0 through 166 flip one q coordinate and packets 167 through 333 flip one s coordinate. The H1 update is a sparse asynchronous rule:

1. compute the exact lag-syndrome field c ,
2. enlarge the current nonzero-lag support by radius 1 to obtain the active mask $\mathcal{M}$,
3. evaluate an exact defect delta Δ_p for every packet $p \in \mathcal{P}$ without materializing the full order-668 matrix,
4. convert Δ_p into a raw packet pressure that rewards defect reduction inside $\mathcal{M}$ and penalizes spill outside $\mathcal{M}$,
5. smooth these pressures over a packet graph consisting of radius-2 same-channel ring edges plus cross-channel same-site edges,
6. subtract refractory penalties, choose locally maximal packets above threshold, and apply at most one packet,
7. update the refractory field and stop on exactness, stagnation, or budget exhaustion.

The full fixed H1 hyperparameter set used in the frontier batch is summarized below:

lag_neighborhood_radius=1	packet_neighborhood_radius=2
same_channel_weight=0.15	cross_channel_weight=0.35
active_lag_bonus=1.0	spill_l1_penalty=0.25
spill_support_penalty=10.0	activation_threshold=0.5
max_active_packets=1	refractory_steps=2
refractory_penalty=0.75	stagnation_limit=8
restart_packet_flips=2	fallback_best_packet=true

Figure 2 summarizes the dataflow.

4.4 Pressure functions and design rationale

Let $\mathcal{M} \subseteq \{1, \dots, \ell - 1\}$ be the current active-lag mask and let $\tilde{c}^{(p)} = c + \Delta_p$ be the exact coefficient vector that would result from firing packet p . The implementation assigns weight

$$w_k = 1 + \beta \mathbf{1}\{k \in \mathcal{L}(c)\}, \quad (15)$$

H1 defect-syndrome CA pipeline in the shared q/s coordinate system

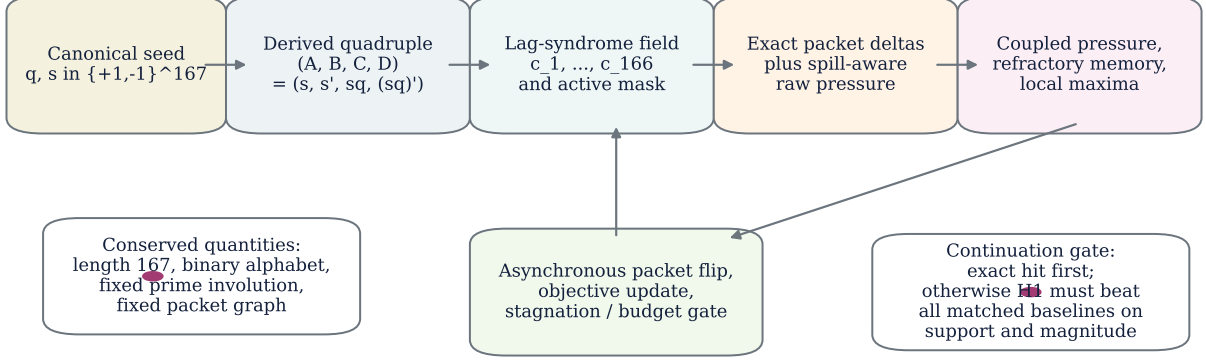


Figure 2: **Implemented H1 pipeline.** The state always remains in the shared q/s representation, so any difference between H1 and the non-CA baselines comes from the update rule rather than from a different search space. Raw packet pressure is computed from exact one-packet defect deltas plus spill penalties; coupled pressure then adds same-channel and cross-channel neighborhood effects plus refractory memory. The continuation gate is external to the update rule: exact hit first, otherwise H1 must strictly beat every matched baseline on support and defect magnitude.

where $\beta = \text{active_lag_bonus}$ and $\mathcal{L}(c) = \{k : c_k \neq 0\}$.

The raw pressure for packet p is

$$\pi_p^{\text{raw}} = \sum_{k \in \mathcal{M}} w_k \left(|c_k| - |\tilde{c}_k^{(p)}| \right) - \lambda_1 \sum_{k \notin \mathcal{M}} \max \left(0, |\tilde{c}_k^{(p)}| - |c_k| \right) - \lambda_0 \max \left(0, \nu(\tilde{c}_{\mathcal{M}^c}^{(p)}) - \nu(c_{\mathcal{M}^c}) \right), \quad (16)$$

where $\lambda_1 = \text{spill_l1_penalty}$, $\lambda_0 = \text{spill_support_penalty}$, and $\nu(v)$ counts nonzero entries of a vector v .

The coupled pressure is then

$$\pi_p = \pi_p^{\text{raw}} + \alpha \sum_{q \in N_{\text{same}}(p)} \pi_q^{\text{raw}} + \gamma \pi_{\text{cross}(p)}^{\text{raw}} - \rho \mathbf{1}\{r_p > 0\}, \quad (17)$$

where $\alpha = \text{same_channel_weight}$, $\gamma = \text{cross_channel_weight}$, and $\rho = \text{refractory_penalty}$.

Each non-obvious design choice is tied to a repository constraint. The compressed representation avoids a 668×668 sign lattice. Spill penalties enforce the prior benchmark rule that prettier defect traces without support control do not count as progress. Refractory memory prevents immediate flip-back oscillation. Restricting `max_active_packets` to 1 keeps the branch in the sparse asynchronous regime rather than collapsing it into a burst heuristic. Because all baselines use the same packet basis, any H1 advantage would have been attributable to the CA update, not to a richer actuator family.

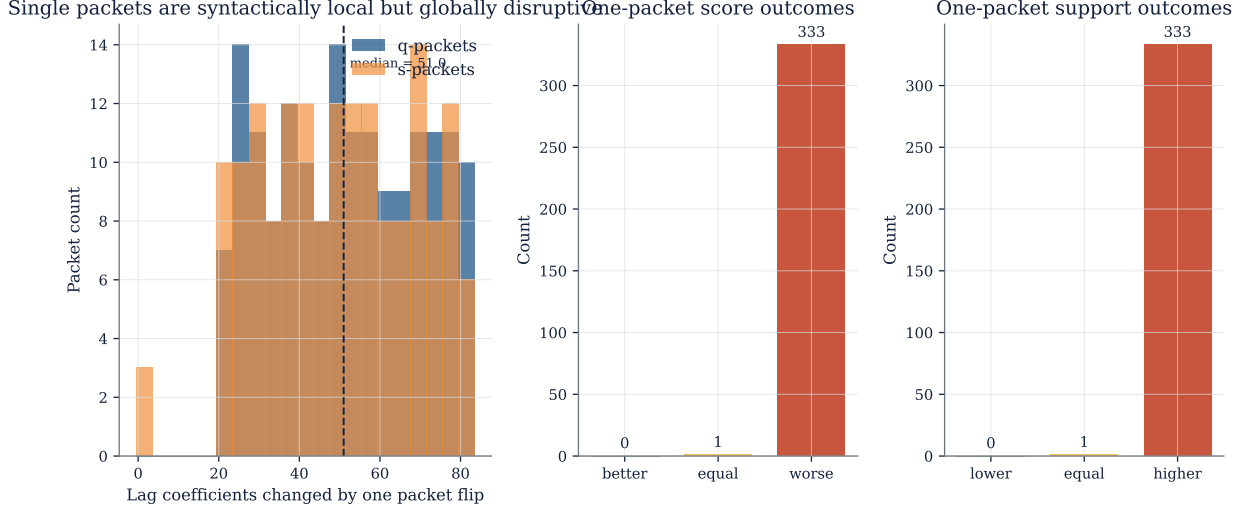


Figure 3: **Direct one-packet locality scan on the canonical frontier seed.** Left: distribution of the number of lag coefficients changed by a single packet flip. The median footprint is 51 lags out of 166, with maximum 83, so a “local” one-packet action is typically global in effect. Middle: exhaustive score comparison over all 334 packets. No packet improves the shared scalar proxy; one packet is neutral. Right: no packet lowers defect support; 333 packets raise it. This finite computation is the direct evidence behind Proposition 5.5.

5 Theoretical Results and Proofs

The present paper does not prove any new existence theorem for Hadamard order 668. Theorems in this section instead serve two narrower purposes: they certify the exact packet-delta machinery implemented by H1, and they convert the repository’s central failure modes into explicit, finite statements.

Proposition 5.1 (Exact single-entry autocorrelation delta). *Let $x \in \{\pm 1\}^\ell$ and fix $i \in \{0, \dots, \ell - 1\}$. Let $x^{(i)}$ denote the sequence obtained by replacing x_i with $-x_i$ and leaving every other coordinate unchanged. Then for every lag $k \in \{1, \dots, \ell - 1\}$,*

$$N_{x^{(i)}}(k) - N_x(k) = -2x_i x_{i+k} \mathbf{1}\{i + k < \ell\} - 2x_{i-k} x_i \mathbf{1}\{i - k \geq 0\}. \quad (18)$$

Proof. Fix k . By Definition 5,

$$N_x(k) = \sum_{t=0}^{\ell-k-1} x_t x_{t+k}.$$

For each summand, compare the term in $N_x(k)$ with the corresponding term in $N_{x^{(i)}}(k)$.

If neither t nor $t + k$ equals i , then the t th summand is unchanged because all coordinates other than i are identical in x and $x^{(i)}$.

There are at most two summands that do involve coordinate i .

Case 1: $t = i$. This case occurs exactly when $i + k < \ell$. The summand in $N_x(k)$ is $x_i x_{i+k}$. The corresponding summand in $N_{x^{(i)}}(k)$ is $(-x_i) x_{i+k}$. The difference is

$$(-x_i) x_{i+k} - x_i x_{i+k} = -2x_i x_{i+k}.$$

Case 2: $t = i - k$. This case occurs exactly when $i - k \geq 0$. The summand in $N_x(k)$ is $x_{i-k}x_i$. The corresponding summand in $N_{x^{(i)}}(k)$ is $x_{i-k}(-x_i)$. The difference is

$$x_{i-k}(-x_i) - x_{i-k}x_i = -2x_{i-k}x_i.$$

No other summands change. Therefore the total change is the sum of the contributions from Case 1 and Case 2, which is exactly (18). This proves the proposition. $\square$

Corollary 5.2 (Exact packet delta for the H1 coordinate system). *Let $(q, s) \in \{\pm 1\}^\ell \times \{\pm 1\}^\ell$ and let Δ_p be the coefficient change caused by firing packet p in the H1 packet basis.*

(a) *If $0 \leq p < \ell$, then packet p flips only q_p , hence*

$$\Delta_p = \delta_{q \odot s}(p) + \delta_{(q \odot s)'}(p), \quad (19)$$

where $\delta_x(i)$ denotes the lag-delta vector from Proposition 5.1 applied to sequence x at index i .

(b) *If $\ell \leq p < 2\ell$, let $i = p - \ell$. Then packet p flips only s_i , hence*

$$\Delta_p = \delta_s(i) + \delta_{s'}(i) + \delta_{q \odot s}(i) + \delta_{(q \odot s)'}(i). \quad (20)$$

Proof. We prove (a); part (b) is analogous but affects four sequences rather than two.

Assume $0 \leq p < \ell$. Firing packet p changes q_p to $-q_p$ and leaves every other coordinate of q and every coordinate of s unchanged. Therefore the derived sequences $A = s$ and $B = s'$ remain unchanged. By contrast, the product sequence $C = q \odot s$ changes exactly at coordinate p because

$$C_p = q_p s_p, \quad C_p^{(p)} = (-q_p) s_p = -C_p.$$

Every other coordinate of C is unchanged because every other coordinate of q is unchanged. The same statement holds for $D = (q \odot s)'$: the prime involution is multiplication by a fixed sign mask, so flipping coordinate p of C flips coordinate p of D and leaves every other coordinate unchanged.

Equation (6) defines the combined coefficient vector as the sum of the four aperiodic autocorrelation vectors. Since only C and D change, the total coefficient delta equals the autocorrelation delta of C plus the autocorrelation delta of D . Proposition 5.1 gives those deltas exactly, yielding the formula in part (a). $\square$

Theorem 5.3 (Complexity of one H1 accepted-state update). *Let ℓ be the sequence length and let the neighborhood radii in H1 be fixed constants independent of ℓ . Then one accepted-state H1 update, including exact coefficient recomputation, packet-pressure recomputation over all 2ℓ packets, coupled-pressure formation, packet selection, and objective evaluation, requires $O(\ell^2)$ arithmetic operations and $O(\ell)$ working memory beyond the logged trace.*

Proof. We account for each stage explicitly.

Stage 1: exact coefficient recomputation. The implementation evaluates four aperiodic autocorrelation vectors of length ℓ . By Definition (5), the k th lag requires $\ell - k$ pairwise products. Summing over all lags gives

$$\sum_{k=0}^{\ell-1} (\ell - k) = \frac{\ell(\ell + 1)}{2}.$$

Therefore one autocorrelation evaluation is $O(\ell^2)$, and the sum of four such evaluations is still $O(\ell^2)$.

Stage 2: active-mask construction. The mask is built from the nonzero-lag support of a vector of length ℓ . This requires scanning at most $\ell - 1$ entries and marking a constant-radius neighborhood around each active lag. Because the radius is fixed, this stage is $O(\ell)$.

Stage 3: raw packet pressures. There are exactly 2ℓ packets. For each packet p , Corollary 5.2 reduces the coefficient delta to the sum of either two or four single-entry autocorrelation deltas. Proposition 5.1 shows that one such delta vector is obtained by looping over the $\ell - 1$ nonzero lags and performing only constant work at each lag. Hence one packet delta costs $O(\ell)$.

After computing Δ_p , the implementation forms $\tilde{c}^{(p)} = c + \Delta_p$, evaluates absolute-value changes on the active mask, and evaluates spill penalties on the complement of the mask. Each of these is a vector operation on arrays of length at most ℓ , hence again $O(\ell)$ per packet. Since there are 2ℓ packets, Stage 3 costs

$$2\ell \cdot O(\ell) = O(\ell^2).$$

Stage 4: coupled pressure and local selection. With fixed same-channel radius and one cross-channel edge per packet, each packet consults only $O(1)$ neighbors. Computing the coupled pressure over all packets is therefore $O(\ell)$, and the local-maxima selection step over the 2ℓ packets is also $O(\ell)$.

Stage 5: objective evaluation and state update. Applying at most one packet is $O(\ell)$ because it copies two length- ℓ arrays and flips one coordinate. Recomputing the objective summary invokes another coefficient computation and is therefore $O(\ell^2)$.

Combining the stages yields

$$O(\ell^2) + O(\ell) + O(\ell^2) + O(\ell) + O(\ell^2) = O(\ell^2).$$

For working memory, the algorithm stores q , s , the coefficient vector, the lag mask, two packet-pressure vectors, and the refractory vector. Their total size is proportional to

$$\ell + \ell + \ell + \ell + 2\ell + 2\ell = O(\ell).$$

The trace can grow with the evaluation budget, but it is a logging artifact rather than a working-memory requirement of one update. This proves the theorem. $\square$

Proposition 5.4 (Budget-bounded termination). *Let B be the global evaluation budget supplied to H1. Then the H1 run halts after at most B objective evaluations.*

Proof. The H1 implementation initializes its evaluation counter at 1, because the starting state is evaluated once before the restart loop. The outer restart loop is guarded by the condition `if evaluations >= evaluation.budget: break`. Inside each restart, the counter is incremented by exactly one whenever a new accepted state is produced. The inner loop is also guarded by the condition `while evaluations < evaluation.budget and stagnation < stagnation_limit`. Consequently no control-flow path can execute an accepted-state update once the counter reaches B . Hence the total number of recorded objective evaluations is at most B . $\square$

Proposition 5.5 (Computational one-packet freeze on the canonical seed). *For the canonical order-668 frontier seed, exhaustive enumeration of all 334 one-packet moves yields:*

- (i) 0 score-improving packets,

- (ii) 1 *score-neutral packet*,
- (iii) 333 *score-worsening packets*,
- (iv) 0 *packets that reduce defect support*.

The median changed-lag footprint of a packet is 51 out of 166 nonzero-lag coordinates, with maximum 83.

Proof. The statement is a complete finite computation. The packet set has cardinality $334 = 2\ell$ with $\ell = 167$. For each packet, the artifact `results/analysis/frontier_locality_scan.json` applies the exact packet flip, recomputes the objective from (10), and records both the score comparison and the changed-lag count from Corollary 5.2. The resulting counts are:

$$\text{better} = 0, \quad \text{equal} = 1, \quad \text{worse} = 333,$$

and for support comparison,

$$\text{lower} = 0, \quad \text{equal} = 1, \quad \text{higher} = 333.$$

The same artifact records changed-lag statistics

$$\text{min} = 0, \quad \text{median} = 51, \quad \text{mean} = 50.78, \quad \text{max} = 83.$$

Because every packet is enumerated exactly once and the packet set is finite, no further search or asymptotic argument is required. This proves the proposition. $\square$

Remark 5.6 (Interpretation of Proposition 5.5). *Proposition 5.5 does not say that every multi-packet route is impossible. It says something narrower but decisive for H1 as implemented: the single-packet actuator library is already frozen at the canonical seed under the shared proxy objective, and a nominally local packet flip usually perturbs about $51/166 \approx 30.7\%$ of the lag coordinates. The only ways left for H1 are neutral drift, coordinated worsening moves, or a new actuator basis.*

Proposition 5.7 (Artifact-certified failure of the first frontier gate). *In the saved canonical frontier batch, H1 fails every continuation condition in the benchmark gate:*

- (a) *H1 records no exact hit.*
- (b) *H1 records no best-objective improvement over the published seed.*
- (c) *H1 does not hold a strict terminal support-and-magnitude advantage over all matched baselines.*

Proof. The frontier summary artifact reports the following H1 values:

$$\text{exact_hit} = \text{false}, \quad \text{best_objective} = (13, 2880, 512),$$

which is exactly the seed objective. This proves (a) and (b).

For (c), the same summary reports H1 terminal restart objectives

$$(33, 2368, 384), \quad (40, 2356, 408), \quad (23, 2216, 384),$$

with median terminal triple (33, 2356, 384). Greedy and simulated annealing each remain at terminal triple (13, 2880, 512). The benchmark gate requires H1 to show lower terminal support *and* lower defect magnitude than every matched baseline when no method reaches exactness. H1’s median terminal support is 33, while the best matched baseline terminal support is 13. Since $33 > 13$, H1 fails the support half of the required strict advantage, regardless of the lower H1 terminal magnitude $384 < 512$. Therefore condition (c) fails. All three continuation conditions are false, so H1 fails the first frontier gate. $\square$

6 Experimental Setup

6.1 Inputs and instances

The study uses three input families.

1. **Exact controls.** Two exhaustive-search controls of lengths 5 and 7 (orders 20 and 28).
2. **Matched control perturbations.** Deterministic $q[0]$ flips of those exact controls, used for the main control batch.
3. **Canonical frontier seed.** The recovered order-668 64-modular seed from Eliahou [7], stored in the repository as exact q/s sequences.

No external dataset is involved. The only stochasticity comes from restart perturbations and, where relevant, method-specific random choices.

6.2 Shared configuration

The main control batch and the canonical frontier batch use the same shared accounting:

$$\begin{array}{ll} \text{evaluation budget} = 80, & \text{requested restarts} = 3, \\ \text{RNG seed} = 17, & \text{restart packet flips} = 2. \end{array}$$

The only method-specific hyperparameters are the H1 coupling and refractory parameters already listed in Section 4, the tabu tenure (5), the simulated-annealing temperature/cooling pair (100, 0.98), and the stochastic-hillclimb sample size (6).

6.3 Reproducibility details

The exact launch commands are frozen in the repository’s reproducibility packet. For the paper, the relevant operational details are:

- all frontier methods read the same `seed_sequences.json` file,
- all methods report through the same harness,
- the run JSONs store top-level *best-over-run* states, not per-restart terminal states,

- the per-restart terminal summaries used in this paper come from the saved aggregation artifacts rather than from `trace[-1]`.

The saved pack does *not* record CPU model, memory size, or OS build as first-class metadata. We therefore report wall-clock times exactly as stored, but we do not treat them as hardware-normalized performance measurements. The paper’s reproducibility emphasis is on seeds, configs, output semantics, and artifact completeness rather than on machine-matched timing.

6.4 Statistics

The study is intentionally small. We therefore avoid pretending that asymptotic uncertainty estimates are stable where they are not. Exact-hit rates are reported together with Wilson 95% confidence intervals only to make the small- n uncertainty explicit. For frontier terminal defect statistics, we report medians together with restart ranges because several methods complete only one restart under the shared budget. For the one-packet locality scan, no confidence interval is needed: the scan is exhaustive over the finite packet set.

7 Results

7.1 Frontier seed geometry and actuator locality

Table 1 and Figures 1–3 establish the starting geometry. The seed is already extremely sparse in correlation-defect space: only 13 of the 166 nonzero-lag coefficients are active. That sparsity is precisely why a local-repair narrative initially looked plausible.

The direct actuator scan overturns the naive locality story. Proposition 5.5 shows that the implemented single-packet library has no improving move on the canonical seed and no move that reduces defect support. Moreover, a nominally local one-packet action is broad in effect, changing a median of 51 lag coordinates and as many as 83. The combination of sparsity in the defect field and breadth in the actuator footprint is the core structural conflict exposed by the paper.

7.2 Matched controls: H1 is executable but not uniquely strong

Figure 4 and Table 2 summarize the control batch. H1 reaches exactness on both solved control perturbations, which is essential because it rules out the trivial explanation that the frontier failure comes from a broken implementation. At the same time, H1 is not uniquely dominant on the controls: greedy and tabu also solve both perturbations, while simulated annealing and stochastic hillclimbing solve one of the two.

This is the correct interpretation of the control batch. It validates the harness and demonstrates executability, but it does not establish a general CA advantage. The controls are tiny, deterministic, and much easier than the order-668 frontier seed.

Table 1: **Canonical frontier seed and direct one-packet locality scan.** The upper block summarizes the published seed recovered from Eliahou [7]. The lower block summarizes the direct exhaustive one-packet scan over the implemented H1 packet basis.

Quantity	Value
Order	668
Sequence length	167
Nonzero correlation lags	13 of 166
Seed objective triple	(13, 2880, 512)
Max off-diagonal Gram defect magnitude	512
Nonzero off-diagonal Gram defects per row	26
Packets scanned exhaustively	334
Improving one-packet moves	0
Neutral one-packet moves	1
Worsening one-packet moves	333
Support-reducing one-packet moves	0
Median changed-lag count	51 of 166
Maximum changed-lag count	83 of 166

Table 2: **Control batch summary.** Wilson intervals are shown only to make the small sample size explicit. Median defect reduction is the reduction in correlation-defect support between the control perturbation and the terminal state.

Method	Exact hits	Wilson 95% CI	Median evaluations	Median defect reduction
H1 CA	2/2	[0.34, 1.00]	12.5	2.5
Greedy	2/2	[0.34, 1.00]	54.0	2.5
Tabu	2/2	[0.34, 1.00]	77.0	2.5
Simulated annealing	1/2	[0.10, 0.91]	63.0	2.0
Stochastic hillclimb	1/2	[0.10, 0.91]	24.5	2.0

7.3 Canonical frontier batch: no exact hit and no best-objective gain

The frontier batch is the decisive experiment. Table 3 and Figure 5 show that no method reaches exactness. More specifically, H1 does not improve the seed best objective at all: its best recorded state is exactly the published seed triple (13, 2880, 512). This verifies Proposition 5.7(b).

What H1 does produce is a characteristic diffusion trajectory. Its three frontier restarts terminate at support values 33, 40, and 23, with median terminal triple (33, 2356, 384). Thus H1 lowers l_1 and maximum defect magnitude only by creating more nonzero lags than the seed. Figure 6 shows this geometrically: every completed restart moves down in defect magnitude only by moving rightward in support. Greedy and simulated annealing, by contrast, simply stay pinned to the seed. Tabu and stochastic hillclimbing wander into worse-support regions without improving the best state.

This is precisely the failure mode anticipated by the benchmark gate. Because no method becomes exact, H1 would need a strict terminal support-and-magnitude advantage over every matched baseline to justify continuation. It does not have that advantage. Its median terminal support 33 is worse than the baseline support 13 achieved by both greedy and simulated annealing. Proposition 5.7 therefore closes the continuation question for H1 on the canonical seed.

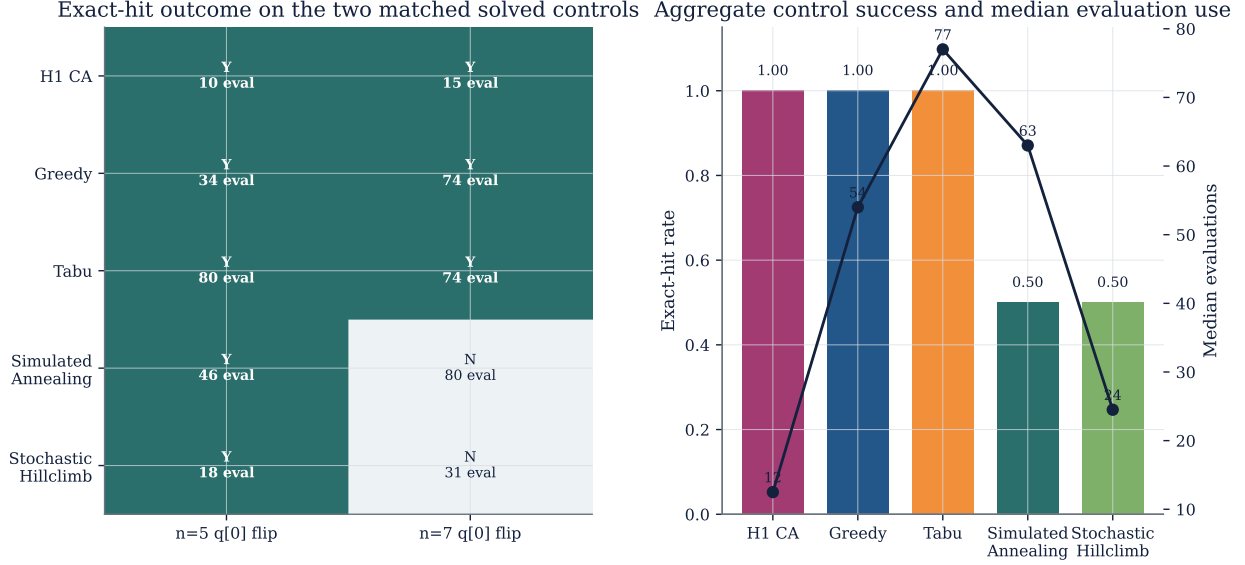


Figure 4: **Matched solved controls.** Left: exact-hit matrix on the order-20 and order-28 control perturbations, with per-run objective evaluations annotated in each cell. Right: exact-hit rate across the two controls together with median evaluation use. H1 reaches exactness on both controls, which rules out a trivial implementation failure, but greedy and tabu do so as well. The control batch therefore validates executability and fair accounting rather than proving a CA-specific advantage.

Table 3: **Canonical frontier batch summary.** Terminal triples are reported as median [min, max] across completed restarts. The exact-hit rate intervals again only expose uncertainty from very small restart counts.

Method	Completed restarts	Exact hits	Wilson 95% CI	Best triple	Terminal median [min, max]	Wall s / evals
H1 CA	3	0/3	[0.00, 0.56]	13/2880/512	33/2356/384 [23/2216/384, 40/2368/408]	2.129/32
Greedy	1	0/1	[0.00, 0.79]	13/2880/512	13/2880/512 [13/2880/512, 13/2880/512]	0.079/80
Tabu	1	0/1	[0.00, 0.79]	13/2880/512	25/3036/520 [25/3036/520, 25/3036/520]	0.090/80
Simulated annealing	1	0/1	[0.00, 0.79]	13/2880/512	13/2880/512 [13/2880/512, 13/2880/512]	0.144/80
Stochastic hillclimb	3	0/3	[0.00, 0.56]	13/2880/512	33/2992/496 [13/2880/480, 42/3040/512]	0.106/26

7.4 Diffusion versus paralysis under parameter variation

The saved H1 sensitivity probe is small but informative. Eight nearby H1 configurations were run on the same canonical seed with one restart and the same budget cap. None produced any best-objective improvement: every variant remained pinned to the seed triple (13, 2880, 512). Figure 6 juxtaposes this parameter paralysis with the actual H1 frontier trajectories. The resulting picture is bifurcated:

- the fixed frontier run moves only by diffusing support, and
- the nearby sensitivity variants avoid diffusion only by staying pinned to the seed.

That is a strong argument against spending more budget on broad H1 sweeps. The problem is not an obviously omitted scalar hyperparameter. It is a structural mismatch between the frontier defect geometry and the current single-packet actuator library.

Canonical frontier batch: seed preservation, support diffusion, and runtime mismatch

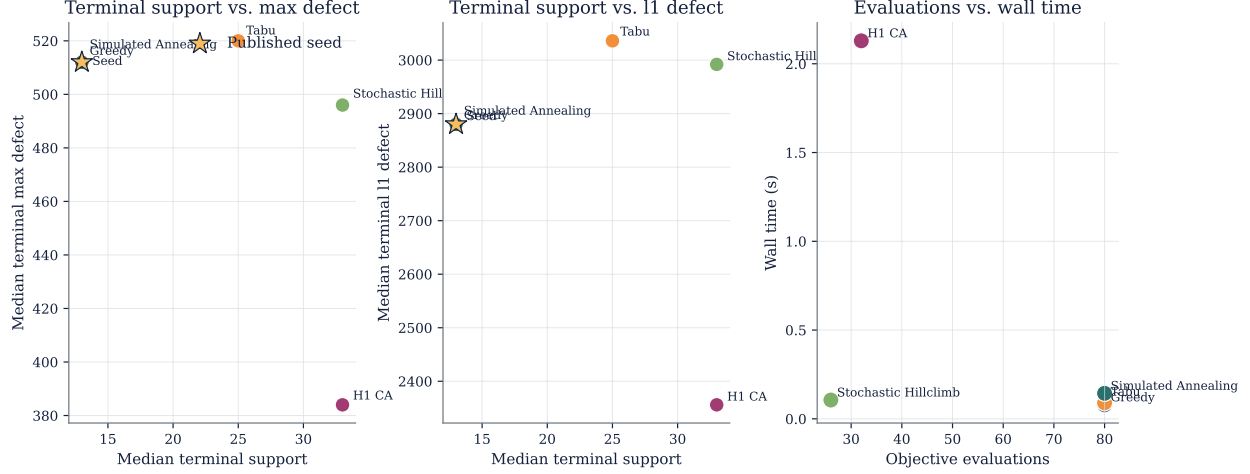


Figure 5: **Canonical frontier batch.** Left and center: each point is one method run from the same published seed under the same nominal budget, restart policy, and packet basis. H1 attains a lower terminal max defect and lower terminal l_1 than the seed only after diffusing support far above the seed support of 13. Greedy and simulated annealing remain pinned to the seed. Right: wall-clock time versus reported objective evaluations. H1 is substantially slower despite fewer objective evaluations, reflecting the heavier internal field computation tracked separately by the CA metadata.

8 Discussion

8.1 What the negative result does establish

The paper establishes three things that are stronger than a generic “we tried CA and it did not work.” First, H1 is executable in the shared coordinate system and solves the two exact controls, so the frontier failure is not a trivial coding defect. Second, the failure is benchmarked against matched non-CA baselines rather than against a straw man. Third, the direct one-packet scan identifies the bottleneck sharply: the current single-packet basis is frozen under the shared proxy objective and broad in its lag footprint.

That combination makes the negative result useful. It rules out a natural but ultimately misleading continuation path: spending more budget on broad H1 rule sweeps while leaving the actuator basis unchanged.

8.2 What the negative result does not establish

The paper does *not* prove that cellular automata are hopeless for order 668. It proves only that the executed H1 branch fails. Several broader claims remain unsupported by the saved pack:

- H1 is not shown to be broadly inferior to all non-CA search.
- The paper does not isolate a causal runtime story beyond the measured fact that H1 is slower in wall time.

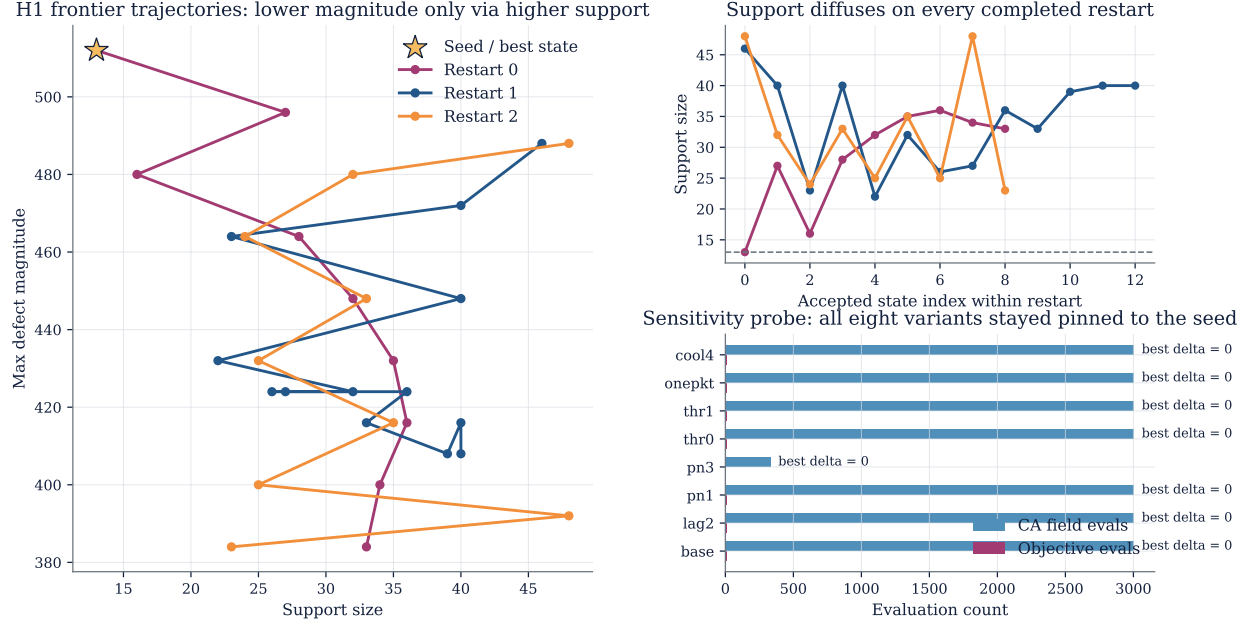


Figure 6: **H1 dynamics and sensitivity.** Left: the three H1 frontier restarts in support–magnitude space. Every completed restart lowers maximum defect only after increasing support above the seed value. Upper right: support as a function of accepted state index within each restart; the dashed line is the seed support. Lower right: eight nearby H1 parameter variants from the saved sensitivity probe. All eight remain pinned to the seed best objective, even though their evaluation counts differ. Together these panels show the repository’s two H1 failure modes: diffusion in the actual frontier run and paralysis under more conservative settings.

- The paper does not certify any statement about full Hadamard equivalence classes, because the stored fingerprint only factors out global q/s sign flips.
- The paper does not justify literature-scale competitiveness claims relative to annealing or SAT+CAS work.

These are important limitations, and they remain explicit because the repository’s verification summary asked for exactly this narrowing.

8.3 Failure mode diagnosis

The central diagnosis is *fake locality*. The seed defect field is sparse, which invites a local-repair narrative, but the actual one-packet actuation is not local in effect. A median one-packet move perturbs 51 of the 166 lag coordinates, and no such move improves the shared proxy objective. Once this is true, the observed frontier trajectories are unsurprising. A monotone or nearly monotone H1 cannot move; a less conservative H1 moves only by spilling defect support into more lags.

This diagnosis also clarifies why the paper does not overinterpret the control successes. Small exact controls can be repaired even when the frontier seed is frozen, because they live in a much easier basin. The control batch therefore validates the scaffold but does not rescue the frontier mechanism.

8.4 Open questions and next steps

The next credible CA step is representation-changing, not parameter-tuning. The repository’s own follow-on hypotheses suggest three classes of move:

1. a composite packet library with smaller lag footprints,
2. a defect–packet influence graph whose locality is topological rather than index-based,
3. a CA-guided hybrid that is explicitly relabeled if it leaks into known structured families.

We do not test those branches here. The present paper is valuable precisely because it says where H1 stopped earning additional budget.

8.5 Societal and scientific implications

Direct societal stakes are limited because the object of study is an abstract combinatorial matrix problem. The more relevant implication is scientific. AI-assisted research pipelines are particularly vulnerable to novelty inflation and to confusing near-solutions with exact results. This repository already contained warning notes about both risks, and the paper preserves them. The exact order-668 problem is not solved here. The value of the present contribution is instead epistemic: it demonstrates how a multi-artifact pipeline can record a negative result cleanly enough to prevent wasted follow-up work and overclaimed frontier narratives.

9 Conclusion

We evaluated a compressed defect-syndrome cellular automaton on the published 64-modular order-668 Hadamard frontier seed under matched same-representation controls. The resulting record is coherent and negative. H1 solves the saved small controls, but on the canonical frontier seed it produces no exact hit, no best-objective gain over the published seed, and no strict continuation-gate advantage over the matched non-CA baselines. An exhaustive one-packet scan explains why: the single-packet actuator library is frozen on the canonical seed and is local only in syntax, not in its lag footprint. The honest conclusion is therefore not that cellular automata solved or nearly solved Hadamard order 668, but that this specific single-packet CA branch has been falsified, and that any future CA attempt must change the representation before it changes the rhetoric.

A Extended Proof Details

A.1 Working-memory accounting for Theorem 5.3

We make the $O(\ell)$ memory claim in Theorem 5.3 explicit. The live arrays in one H1 update are:

- two state arrays q and s of length ℓ ,
- one coefficient vector of length ℓ ,

- one Boolean lag mask of length ℓ ,
- one refractory vector of length 2ℓ ,
- two packet-pressure vectors of length 2ℓ ,
- a constant number of temporary vectors of length at most ℓ inside one packet evaluation.

Summing lengths gives at most a constant multiple of ℓ . The trace can grow with the evaluation budget, but that is output storage, not working storage.

A.2 Consistency of the frontier H1 internal-work counter

The saved H1 frontier run reports `ca_field_evaluations`= 10354. The implementation increments this counter by exactly $2\ell = 334$ whenever a nonempty lag mask triggers a full packet-pressure sweep. The same run stores a trace of length 31, and every saved H1 frontier trace entry has a nonempty active-lag mask. Therefore

$$31 \times 334 = 10354,$$

which matches the recorded metadata exactly. This does not prove that field evaluation is the whole runtime story, but it does show that the internal-work counter is arithmetically self-consistent.

B Additional Experimental Tables

B.1 Per-run control results

Table 4: **Per-run control outcomes.** These are the ten saved runs underlying Figure 4 and Table 2.

Seed	Method	Exact hit	Terminal support	Max defect	Evaluations
<code>control_n5_q0</code>	H1 CA	yes	0	0	10
<code>control_n5_q0</code>	Greedy	yes	0	0	34
<code>control_n5_q0</code>	Simulated annealing	yes	0	0	46
<code>control_n5_q0</code>	Stochastic hillclimb	yes	0	0	18
<code>control_n5_q0</code>	Tabu	yes	0	0	80
<code>control_n7_q0</code>	H1 CA	yes	0	0	15
<code>control_n7_q0</code>	Greedy	yes	0	0	74
<code>control_n7_q0</code>	Simulated annealing	no	1	4	80
<code>control_n7_q0</code>	Stochastic hillclimb	no	1	4	31
<code>control_n7_q0</code>	Tabu	yes	0	0	74

B.2 Baseline smoke batch

The smoke batch does not bear on order 668, but it does validate that the non-CA baselines share one representation and one exactness definition. On the order-28 smoke seed, all four non-CA baselines reach exactness. Simulated annealing ends at a different exact fingerprint than the

exhaustive control fingerprint, which confirms that the harness accepts exactness independently of matching a single canonical exemplar.

Table 5: **Smoke-batch exactness check on the non-exact order-28 seed.**

Method	Exact hit	Evaluations
Greedy	yes	74
Tabu	yes	74
Simulated annealing	yes	80
Stochastic hillclimb	yes	27

C Implementation Details and Hyperparameters

C.1 Full frontier configuration

Table 6: **Saved frontier configurations.**

Item	Value
Shared evaluation budget	80
Shared requested restarts	3
Shared RNG seed	17
Shared restart packet flips	2
H1 lag neighborhood radius	1
H1 packet neighborhood radius	2
H1 same-channel weight	0.15
H1 cross-channel weight	0.35
H1 active-lag bonus	1.0
H1 spill l_1 penalty	0.25
H1 spill support penalty	10.0
H1 activation threshold	0.5
H1 max active packets	1
H1 refractory steps	2
H1 refractory penalty	0.75
H1 stagnation limit	8
Tabu tenure	5
Simulated annealing temperature	100.0
Simulated annealing cooling	0.98
Stochastic hillclimb sample size	6

C.2 Output semantics

The top-level run files store the *best-over-run* state, not the last accepted state of the final restart. This matters especially for H1, whose best frontier state is the unchanged published seed while its restart terminals diffuse to larger support. Any attempt to read `trace[-1]` as the final experimental outcome would therefore be incorrect. The paper consistently uses the saved aggregation tables for restart-terminal summaries.

C.3 Exact rerun commands

The reproducibility packet already freezes the exact commands. The minimal paper-facing summary is:

- run `python3 scripts/run_h1_ca.py` with the H1 config and the seed file,
- run `python3 scripts/run_baseline.py` with each baseline config and the same seed file,
- regenerate figures and the direct one-packet locality artifact with the artifact generator script `scripts/generate_paper_artifacts.py`.

D Hyperparameter Sensitivity

The saved sensitivity probe covers eight nearby H1 configurations on the canonical seed with one restart each. Their main shared finding is negative: every variant remains pinned to the seed best objective. Table 7 records the saved counts.

Table 7: **Saved H1 sensitivity probe on the canonical frontier seed.** Every variant keeps the seed best objective unchanged.

Variant	Lag radius	Packet radius	Threshold	Objective evals	CA field evals
base	1	2	0.5	10	3006
lag2	2	2	0.5	10	3006
pn1	1	1	0.5	10	3006
pn3	1	3	0.5	2	334
thr0	1	2	0.0	10	3006
thr1	1	2	1.0	10	3006
onepkt	1	2	0.5	10	3006
cool4	1	2	0.5	10	3006

References

- [1] F. J. Aragón Artacho, J. Borwein, and Matthew K. Tam. Douglas–rachford feasibility methods for matrix completion problems. *The ANZIAM Journal*, 2013. doi: 10.1017/S1446181114000145. URL <https://www.semanticscholar.org/paper/57fca051597c048ba77b11e9efcafd5741ea4dbc>.
- [2] Curtis Bright, I. Kotsireas, and Vijay Ganesh. A sat+cas method for enumerating williamson matrices of even order. *AAAI Conference on Artificial Intelligence*, 2018. doi: 10.1609/aaai.v32i1.12203. URL <https://www.semanticscholar.org/paper/eec5b2d83d3853c24cf070b4701fc2c86b594222>.
- [3] Curtis Bright, Dragomir Z. Dokovic, I. Kotsireas, and Vijay Ganesh. The sat+cas method for combinatorial search with applications to best matrices. *Annals of Mathematics and Artificial Intelligence*, 2019. doi: 10.1007/s10472-019-09681-3. URL <https://www.semanticscholar.org/paper/74aeec834364032b323d56b8a1c03c798fcd2844>.

- [4] Matteo Cati and Dmitrii V. Pasechnik. A database of constructions of hadamard matrices. *arXiv.org*, 2024. doi: 10.48550/arXiv.2411.18897. URL <https://www.semanticscholar.org/paper/9f2616d562a9810dfc55db9f9acf606c2ef62296>.
- [5] S. Eliahou and M. Kervaire. Modular sequences and modular hadamard matrices. *Journal of Combinatorial Designs*, 2001. doi: 10.1002/JCD.1007. URL <https://www.semanticscholar.org/paper/74c6c98807c49280d7dbdc8c935f625788248b03>.
- [6] S. Eliahou and M. Kervaire. A survey on modular hadamard matrices. *Discrete Mathematics*, 2005. doi: 10.1016/j.disc.2005.02.021. URL <https://www.semanticscholar.org/paper/b0bc1d196cbb14824aebcbb2410226ff2688d3d>.
- [7] Shalom Eliahou. A 64-modular hadamard matrix of order 668. *Australasian Journal of Combinatorics*, 93(2):422–427, 2025. URL https://ajc.maths.uq.edu.au/pdf/93/ajc_v93_p422.pdf.
- [8] M. Gadouleau, Luca Mariot, and S. Picek. Bent functions from cellular automata. *IACR Cryptology ePrint Archive*, 2020. URL <https://www.semanticscholar.org/paper/792b2568f1335f14e5282dd837943d109fbb4b85>.
- [9] Luca Mariot, M. Gadouleau, E. Formenti, et al. Mutually orthogonal latin squares based on cellular automata. *Designs, Codes and Cryptography*, 2019. URL <https://www.semanticscholar.org/paper/2610be30aa699bdc403fe1e417a472e47a2800a7>.
- [10] A. B. Suksmono. Finding a hadamard matrix by simulated quantum annealing. *Entropy*, 2018. doi: 10.3390/e20020141. URL <https://www.semanticscholar.org/paper/d98246df005a52ff9a5e99b6c5f548c1def1ba12>.
- [11] A. B. Suksmono and Yuichiro Minato. Finding hadamard matrices by a quantum annealing machine. *Scientific Reports*, 2019. doi: 10.1038/s41598-019-50473-w. URL <https://www.semanticscholar.org/paper/c42e6a8a231cd8cd638ff953b6aa7b539c4596e9>.
- [12] A. B. Suksmono and Yuichiro Minato. Quantum computing formulation of some classical hadamard matrix searching methods and its implementation on a quantum computer. *Scientific Reports*, 2022. doi: 10.1038/s41598-021-03586-0. URL <https://www.semanticscholar.org/paper/1fd50d1147b7684686e533886b44a9e92ac2e807>.
- [13] Michail-Antisthenis I. Tsompanas, Nikolaos I. Dourvas, K. Ioannidis, G. Sirakoulis, R. Hoffmann, and A. Adamatzky. Cellular automata applications in shortest path problem. *arXiv.org*, 2017. doi: 10.1007/978-3-319-77510-4_8. URL <https://www.semanticscholar.org/paper/4b85674750c1ad93a8d26427e52e7bec82d7e392>.